

Chunk Scaled Dot Kkt 算子开发和优化分享

分享人：赵润田

目录

Part 1 算子功能和公式

Part 2 算子实现

Part 3 算子性能优化

Part 4 算子瓶颈分析

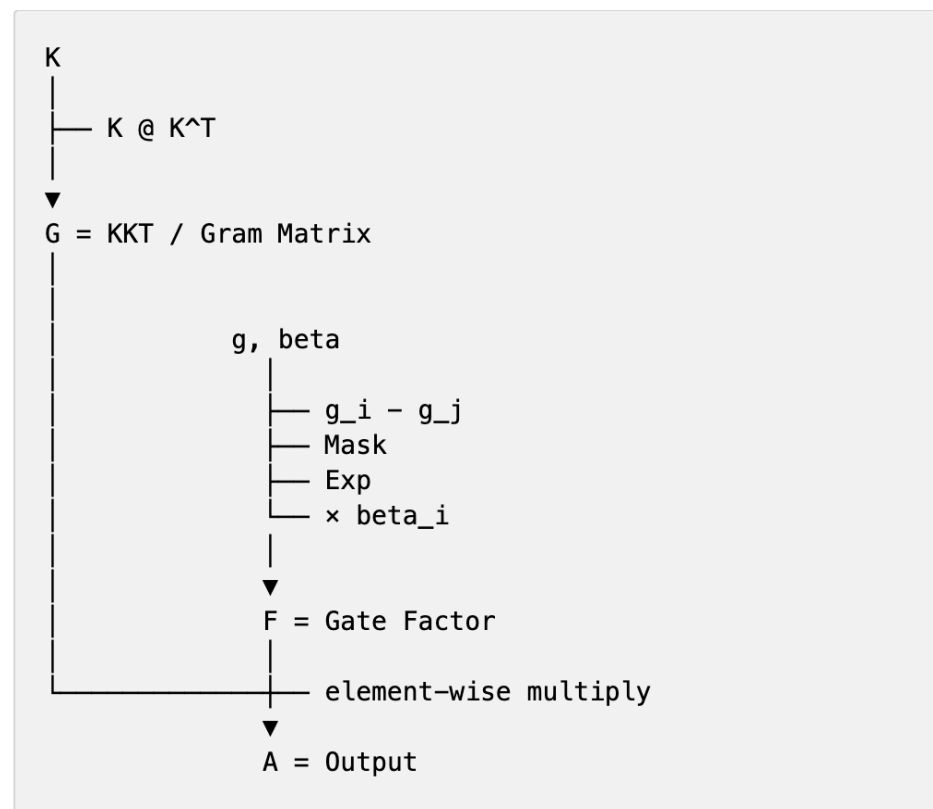
Part 5 一些反馈

一、算子功能和公式

算子公式:

$$A_{ij} = \begin{cases} \beta_i \exp(g_i - g_j) (k_i^\top k_j), & g_i - g_j < 0, \\ 0, & g_i - g_j \geq 0. \end{cases}$$

算数流程示意:



1. KKT

输入：

$$K \in \mathbb{R}^{L \times d}, \quad L \leq 64, \quad d \in \{128, 256\}$$

计算：

$$G = K K^T$$

逐元素：

$$G_{ij} = k_i^T k_j$$

Full chunk 时：

$$(64 \times d)(d \times 64) \rightarrow 64 \times 64$$

2. Gate Difference

矩阵形式：

$$D_{ij} = g_i - g_j$$

$$D = g \mathbf{1}^T - \mathbf{1} g^T$$

$$D = \begin{bmatrix} g_1 - g_1 & g_1 - g_2 & g_1 - g_3 & \cdots & g_1 - g_L \\ g_2 - g_1 & g_2 - g_2 & g_2 - g_3 & \cdots & g_2 - g_L \\ g_3 - g_1 & g_3 - g_2 & g_3 - g_3 & \cdots & g_3 - g_L \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ g_L - g_1 & g_L - g_2 & g_L - g_3 & \cdots & g_L - g_L \end{bmatrix}$$

3. Gate Factor

$$F_{ij} = \beta_i \cdot e^{g_i - g_j} \cdot \mathbf{1}[g_i - g_j < 0]$$

4. Output

逐元素相乘：

$$A = G \odot F$$

因此：

$$A_{ij} = (k_i^T k_j) \cdot \beta_i \cdot e^{g_i - g_j} \cdot \mathbf{1}[g_i - g_j < 0]$$



一、算子功能和公式：具体输入和输出

输入/输出	参数名	类型	维度形状	支持数据类型	数据格式	描述
INPUT(必选)	k	tensor	[B, T, Hg, K]	bfloat16	ND	Key特征张量
INPUT(必选)	beta	tensor	[B, H, T]	bfloat16	ND	缩放系数
INPUT(必选)	g_cumsum	tensor	[B, H, T]	float32	ND	门控/累积项
INPUT(必选)	chunk_offsets	tensor	[NT+1]	int32	ND	变长模式下的chunk索引
INPUT(必选)	chunk_size	scalar	零维	int32	ND	T轴切分的chunk大小
OUTPUT(输出)	A	tensor	[B, H, T, BT]	float32	ND	输出张量

其中， `chunk_offsets[i]` 表示第 `i` 个chunk在T轴上的起始位置， `chunk_offsets[i+1] - chunk_offsets[i]` 为该chunk的有效长度 `chunklen` 。递增序列，最后一个数值等于T， `chunklen<=chunk_size` 。



一、算子功能和公式：输入约束（范围）

•参数解释

- B: 批次大小, 当前B=1
- T: 序列长度
- H: 多头注意力中的头数 (有多少组不同的 beta 以及 g_cumsum)
- Hg: 多头注意力分组数 (有多少组不同的K)
- BT: 分块大小chunk_size, 当前BT=64
- K: 每个注意力头的键向量维度
- NT: 序列chunk数
- chunk_offsets: 分块索引列表, 如 [0, 64, 128, ..., T], 单调递增且递增大小 $\leq$ BT
- 维度取值范围(均为正整数):
 - B: 1
 - T: 包含序列个数[1,4], 单个序列长度[1k, 64K], T长度范围[1k, 256k]
 - H: [1, 128]
 - Hg: [1, 32], Hg是H的因子
 - K: 128和256
 - chunk_offsets: 单调递增, 举例: T = 300, 其中包含序列长度3, cu_seqlens = [0, 100, 240, 300] 则chunk_offsets = [0, 64, 100, 164, 228, 240, 300]
 - chunk_size: 64

二、算子实现：实现一个低风险的baseline

任务分配： 任务不是 query head(H)，而是 chunk × KV head(H_g)

定义：

$$R = numRepeat = \frac{H}{H_g} \quad task = (chunk, KVhead)$$

一个 task 计算一个 64×64 KKT block

一个 KKT block 会被 R 个 query heads 复用

$K \in \{128, 256\}$

$chunkLen \in [1, 64]$

存在 full chunk 和 tail chunk

存在多 chunk、多 KV head、多 query head、变长序列

(chunk, kvHead)



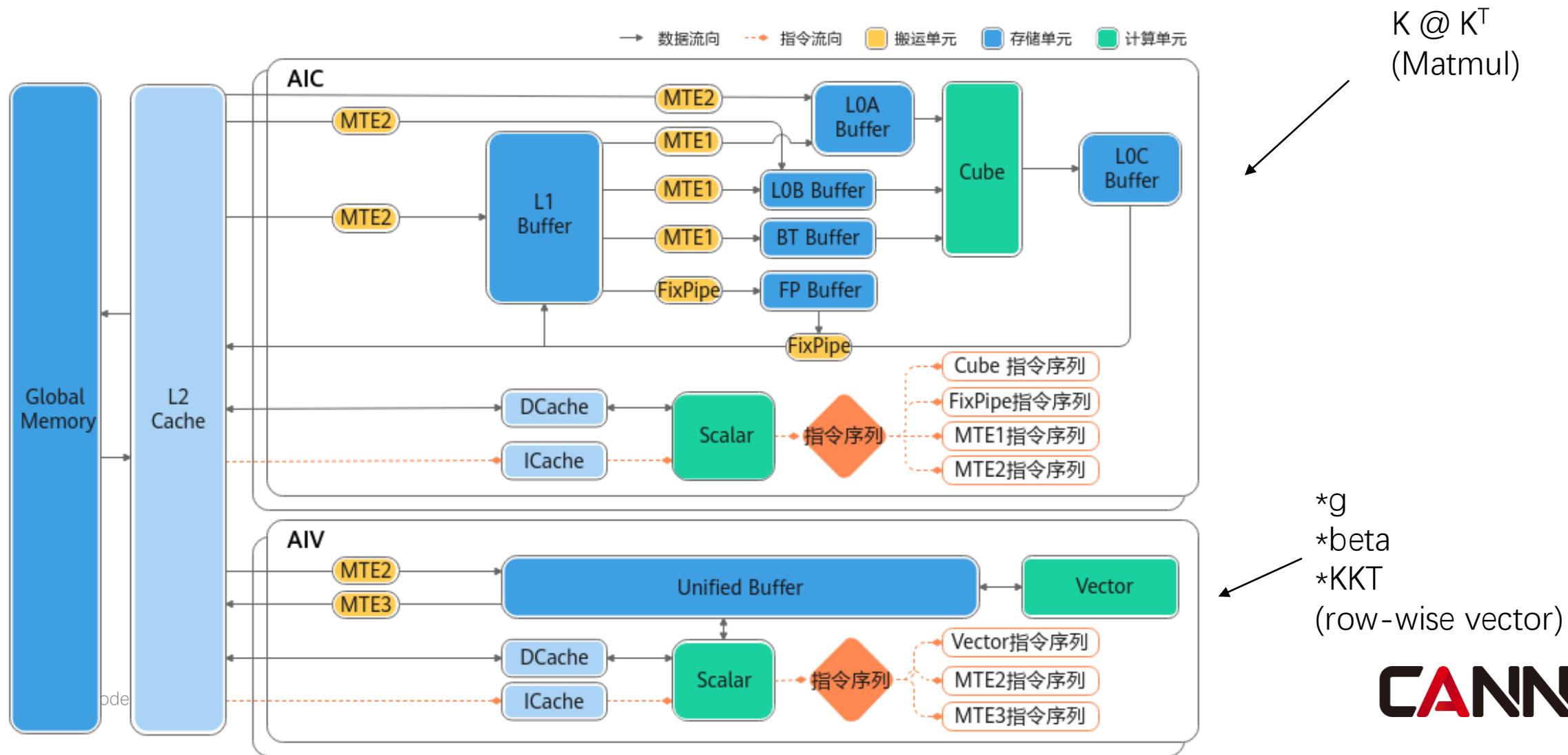
一个 KKT



├── query head 0: g0 / beta0
├── query head 1: g1 / beta1
├── query head 2: g2 / beta2
└── ...

二、算子实现：实现一个低风险的baseline

Arch22（910B等芯片），从底层架构出发，将计算任务分配到最适配的硬件上去



二、算子实现： 具体算数 流程

CANN

<https://gitcode.com/cann>

AIC

$$K \cdot K^T = \left\{ \left[\begin{array}{c} \text{ } \end{array} \right] \cdot \left[\begin{array}{c} \text{ } \end{array} \right]^T \right\} = \left[\begin{array}{c} \text{ } \end{array} \right]$$

$k = 128/256$

L (通常为 64)

64×64

KK^T (64 x 64)

AIV

① CopyIn : 获取 g & β & KK^T

② 初始化:

Duplicate (negInfLocal, -FLT_MAX, 64);

Muls (negGLocal, glocal, -1.0f, 64);

③ 针对每一行: for each "row"

float $g_i = gLocal \cdot GetValue(row);$

float $\beta = \beta Local \cdot GetValue(row);$

计算差值: Adds (outputRow, negGLocal, g_i , 64); // outputRow[j] = $g_i - g_j$

产生Mask: CompareScalar (mask, output, 0, LT, 64); // mask[j] = ($g_i - g_j < 0$)

非mask位置
设为极小值: Select (outputRow, mask, outputRow, negInfLocal);

$$\Rightarrow gate_j = \begin{cases} g_i - g_j & g_i - g_j < 0 \\ -FLT_MAX & otherwise \end{cases}$$

指数获取: Exp (outputRow, outputRow, 64);

$$x_j = \begin{cases} \exp(g_i - g_j) & g_i - g_j < 0 \\ 0 & otherwise \end{cases}$$

乘 β & KK^T : Muls (outputRow, outputRow, β , 64);

Mul (outputRow, KKTRow, outputRow, 64);

二、算子实现：踩过的一個坑——尾部处理

根据chunk_offsets, 部分chunk的L不足64, 会造成一些尾部问题, 如果直接按通用方式处理, 会有正确性的bug。

解决方案

compact [L × L]

1. 假设: L = 37

我们所需要的

row 0 [37 valid][27 pad]
row 1 [37 valid][27 pad]
row 2 [37 valid][27 pad]

stride = 64

实际 Matmul 输出

row 0 [----- 37 -----]
row 1 [----- 37 -----]
row 2 [----- 37 -----]

stride = 37

2.

单纯32B (UB的需求) 对齐并不够, 这里我们的数据是float32, 32B对齐对应的是4个float32, 所以datapad后是
[----- 37 valid -----][3 pad]
<----- 40 ----->

<https://gitcode.com/cann>

row0 [---- L ----]
row1 [---- L ----]
row2 [---- L ----]

- |
- | ① UB 清零
- | ② 只搬 L 个有效值
- | ③ 按 32B 做 DataCopyPad



AIV Working UB

row0 [--L--][padding.....]
row1 [--L--][padding.....]
row2 [--L--][padding.....]

<----- 64 ----->

CANN

三、算子性能优化：常规的一些思路

- 1、通过tiling（任务切分）尽量能把所有的AI core分配满（但是要注意，对于中小的case，并不一定用越多的core就会更快，要考虑通信和数据等待的开销）。
- 2、根据计算性质分配到合适的执行单元（AIC/AIV）
- 3、优化数据搬运和存储布局
- 3、利用好AIV的特性（Vector适配的API），想办法怎么能把scalar的一些计算方式通过转变为vector的计算方式
- 4、AIC可以通过低阶手写MMAD，降低固定开销，获得一定的加速
- 5、调整流水、buffer，将尽可能多的流水并行工作（当然也需要考虑到数据依赖的性质）

.....

三、算子性能优化：工具和参考资料

Profile工具：

msprofile：真实上板，且有真实的较为详细的计算时间数据，以及数据搬运大小的数据

msprofile simulator：仿真分析，能看到具体的流水、时间占比、指令数量等…

参考资料：

官方算子库：

<https://gitcode.com/cann/ops-transformer>

<https://gitcode.com/cann/ops-nn>

<https://gitcode.com/cann/ops-math>

官方教学：<https://gitcode.com/cann/cann-learning-hub>

高性能算子示例：<https://gitcode.com/cann/cann-samples>

赛事冠军开源代码：<https://gitcode.com/cann/cann-ops-competitions>



Block ID ⓘ: 0

Show As ⓘ: Bandwidth(GB/s)

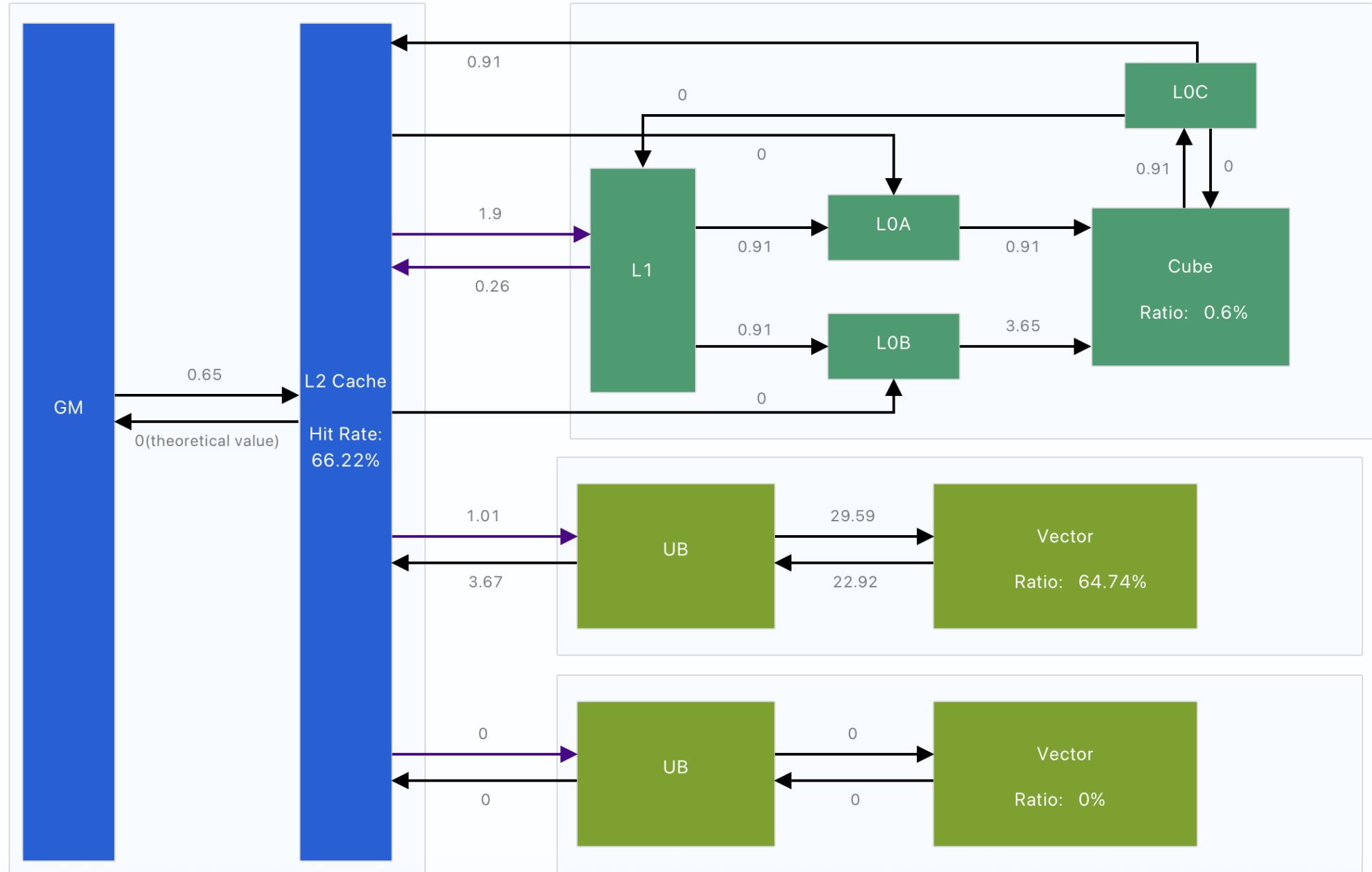
T=10016

H=8

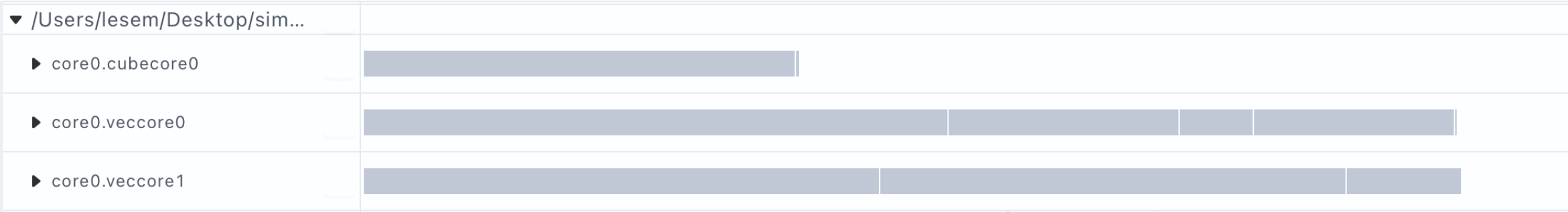
Hg=2

K=128

Duration=
269 us



三、算子性能优化： V2: 转换成AIC/AIV完全分离的架构



AIV中占据cycle数前十的指令

105	0x140e8760	VECTOR	BAR PIPE:VEC	256
1929	0x140e6e28	FLOWCTRL	WAIT_FLAG_DEVI U...	1
184	0x140e8574	VECTOR	BAR PIPE:VEC	256
102	0x140e8754	SCALAR	ST_XD_XN_IMM dty...	256
132	0x140e87cc	SCALAR	ST_XD_XN_IMM dty...	246
110	0x140e8774	VECTOR	VMULS XD:X14,XN:...	256
114	0x140e8784	VECTOR	MOVEMASK XN:X26...	256
115	0x140e8788	VECTOR	MOVEMASK XN:X26...	256
139	0x140e87e8	VECTOR	VMULS XD:X15,XN:...	246

AIC:

K @ K^T

→ FP32 KKT

→ 写入 GM user workspace

→ SyncAll<false>()

AIV:

SyncAll<false>()

→ 从 GM user workspace 读取 KKT

→ gate / beta / exp / mask / Mul

→ 写回 output



三、算子优化：V2优化：在 AIC/AIV 独立阶段做 Vector 针对性优化

Brcb

V1:通过逐行的GetValue来获取g和beta

V2:通过Brcb的vector指令，将g和beta向量化，从而可以以更少的scalar指令获取diff以及乘上beta

</> C++

```
Muls(negGLocal, gLocal, -1.0f, 64);

for (int row = 0; row < rowCount; ++row) {
    float gi = gLocal.GetValue(rowStart + row);

    Adds(
        outputLocal[row * 64],
        negGLocal,
        gi,
        64);
}
```

```
Brcb(scratch, g, 8, {1, 8});

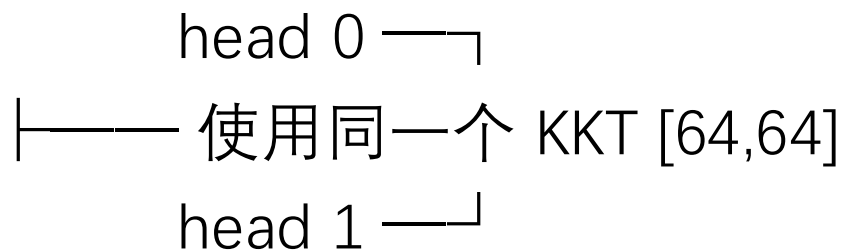
Add(
    out,
    negG,
    scratch,
    64,
    64,
    {1, 1, 0, 8, 0, 1});
```

三、算子优化：V2优化：在 AIC/AIV 独立阶段做 Vector 针对性优化

Double-head

V1:每个AIV在一次任务中只搬取和计算一个query head中的g和beta

实际：



如果一个AIV需要处理2个及以上的head，则可以同时处理两个，可以减少一半的PiepBarrier

<https://gitcode.com/cann>

```
constexpr int HEAD_BATCH = 2;

for (int pair = 0; pair < pairCount; ++pair) {
    // 一个 MTE2 batch 搬入两个 head 的 g/beta
    PrefetchGateAndBetaBatch(
        firstPairHead,
        start,
        chunkLen,
        HEAD_BATCH);

    auto gPair =
        gQueue.DeQue<float>();

    auto betaPair =
        betaQueue.DeQue<bfloat16_t>();

    auto out0 =
        outputHead0Queue.AllocTensor<float>();

    auto out1 =
        outputHead1Queue.AllocTensor<float>();

    // 同时准备两个 head 的 Factor
    PrepareHeadPairFactor(
        gPair,
        betaPair,
        rowStart,
        rowCount,
        out0,
        out1);

    // 两个 Factor 复用同一份 KKT
    Mul(out0, kktLocal, out0, activeElems);
    Mul(out1, kktLocal, out1, activeElems);

    // 分别写出两个 head
    CopyOut(firstPairHead, out0);
    CopyOut(firstPairHead + 1, out1);
}
```

Mix Block Detail:

Block ID	Cube0 Duration (μs)	Vector0 Duration (μs)	Vector1 Duration (μs)
0	21.151667	123.217781	123.283890
1	21.131666	123.337776	123.277222
2	21.098888	123.177780	123.101112
3	20.929445	122.994446	123.176666
4	21.004444	122.137779	122.853889
5	20.874445	121.959999	122.165558



Block ID	Cube0 Duration (μs)	Vector0 Duration (μs)	Vector1 Duration (μs)
0	21.409445	65.888885	65.642776
1	21.337223	65.896111	65.593330
2	21.415556	65.961670	66.192223
3	21.352222	66.051666	65.827774
4	21.202223	65.008331	65.569443
5	21.158333	65.048889	65.638336

三、算子优化：V3优化： 做出AIV/AIC流水，隐藏AIC时间

Block ID	Cube0 Duration (μs)	Vector0 Duration (μs)	Vector1 Duration (μs)
0	21.409445	65.888885	65.642776
1	21.337223	65.896111	65.593330
2	21.415556	65.961670	66.192223
3	21.352222	66.051666	65.827774
4	21.202223	65.008331	65.569443
5	21.158333	65.048889	65.638336

AIC KKT0 KKT1 KKT2 KKT3.....
AIV Vec0 Vec1 Vec2 Vec3.....

T=10016 H=8 Hg=2 K=128 AIC: AIV duration = 1 : 3

最理想的流水状况下，最多可以节约大约33%的时间



三、算子优化：V3优化：双slot实现AIC/AIV并行

双 slot：实现 AIC/AIV 的 Producer–Consumer 流水并行

time —————>

slot0:

AIC KKT0 —READY—> AIV Load KKT0 —FREE—> AIC KKT2

slot1:

AIC KKT1 —READY—> AIV Load KKT1 —FREE—> AIC KKT3

“双 slot”本身不是直接“让 AIC 和 AIV 同时算”，而是通过两个 workspace slot 交替复用，避免 producer 和 consumer 因为同一块中间结果存储而互相阻塞，从而实现：

$AIC(t+1) \parallel AIV(t)$

三、算子优化：V3优化：AIV内部流水优化

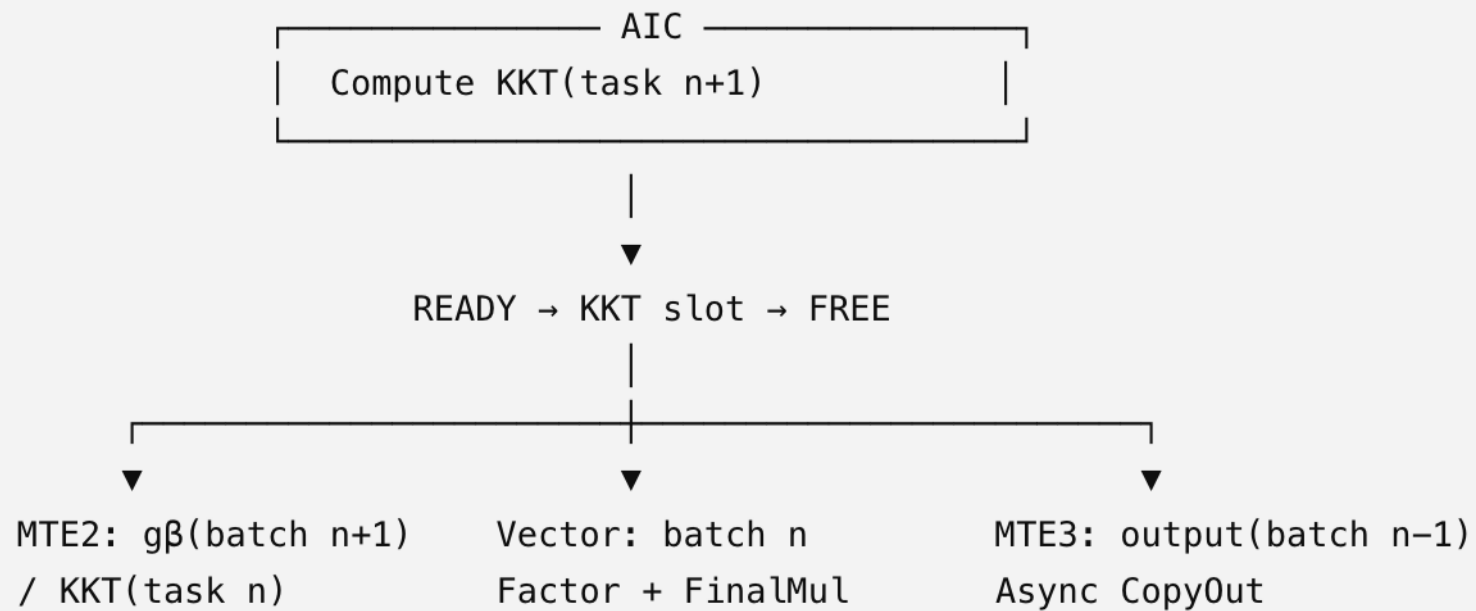
原始串行
Wait READY
→ Load KKT
→ Load g / beta
→ Build Factor
→ Mul KKT
→ CopyOut



优化后：

1. 等待 READY 前
预取当前 task 第一批 g / beta
2. KKT 从 workspace 搬入 UB 时
构造第一批 KKT-independent Factor
3. KKT 搬运完成后
尽早发送 FREE，释放共享 slot
4. 当前 batch 做 Factor / FinalMul 时
不阻塞上一批MTE3的取回
5. 当前 batch 计算或输出期间
预取下一批 g / beta
6. 达到 output retirement window 后
再统一等待 MTE3 并释放 VECOUT Tensor

大致流水图



稳态:

KKT(n+1) || gβ(n+1) || Compute(n) || Output(n-1)

三、算子优化：V4优化：低阶手写MMAD

动机：

M = N = 64 固定（除尾部一些特殊情况）

K 只有 128 / 256

Gram Matrix 结构固定： $K \times K^T$

高层 Matmul 通用性强，但存在调度和初始化开销

小 case 中固定开销更明显

数据流

GM K

↓

L1 / A1 / B1

↓

L0A / L0B

↓

MMAD

↓

CO1 / L0C

↓

Fixpipe

↓

FP32 KKT workspace

```
</> C++
```

```
template<int K_DIM>
__aicore__ void ComputeGramMmad(Task t, int slot) {
    ClearL1Tile();

    LoadKTileToL1(t);      // tail 时 full-pad

    ClearC01();

    for (int part = 0; part < K_DIM / 64; ++part) {
        LoadL0A(part);
        LoadL0B(part);

        Mmad(/*accumulate=*/part != 0);
    }

    FixpipeToWorkspace(slot);
}
```

三、算子优化：V4优化：低阶手写MMAD导致的correctness问题

1.最初写法，没有注意到在进行mmad的时候，下一个K-part可能正在写入L0A/L0B

GM K
├──→ A1
└──→ B1

for each K-part:

L1 → L0A / L0B

MMAD

继续下一个 K-part

最后：

L0C → Fixpipe → Workspace

2.第一轮改进： L0A/L0B做 Ping-Pong

L0A[0] → 独立 event

L0A[1] → 独立 event

L0B[0] → 独立 event

L0B[1] → 独立 event

(每块物理buffer的ownership独立管理)

MMAD(part0) 读 stage0

同时

MTE1(part1) 写 stage1

注意：

1. K=256时，有4个part，所以需要分出4个独立的stage

2. Task 边界显式结束 L0 的使用，并重新初始化 ownership event (否则在长任务下会失效)

三、算子优化：V5优化：任务分配优化——让同一组连续处理相邻chunk

1、之前的任务分配: Round-Robin

假设：N_chunk=8, Hg=2, blockDim=4

逻辑上的 taskId:

0	1	2	3	...
C0H0	C0H1	C1H0	C1H1	

原始分配

Group 0 : C0H0 → C2H0 → C4H0 → C6H0

Group 1 : C0H1 → C2H1 → C4H1 → C6H1

Group 2 : C1H0 → C3H0 → C5H0 → C7H0

Group 3 : C1H1 → C3H1 → C5H1 → C7H1

2、之后的任务分配: 连续性

物理内存上:

H0 : C0 → C1 → C2 → C3 → C4 → C5 → C6 → C7

H1 : C0 → C1 → C2 → C3 → C4 → C5 → C6 → C7

连续性分配

Group 0 : H0C0 → H0C1 → H0C2 → H0C3

Group 1 : H0C4 → H0C5 → H0C6 → H0C7

Group 2 : H1C0 → H1C1 → H1C2 → H1C3

Group 3 : H1C4 → H1C5 → H1C6 → H1C7

```
// totalTasks = numChunks * Hg
// blockDim   = AIC group 数

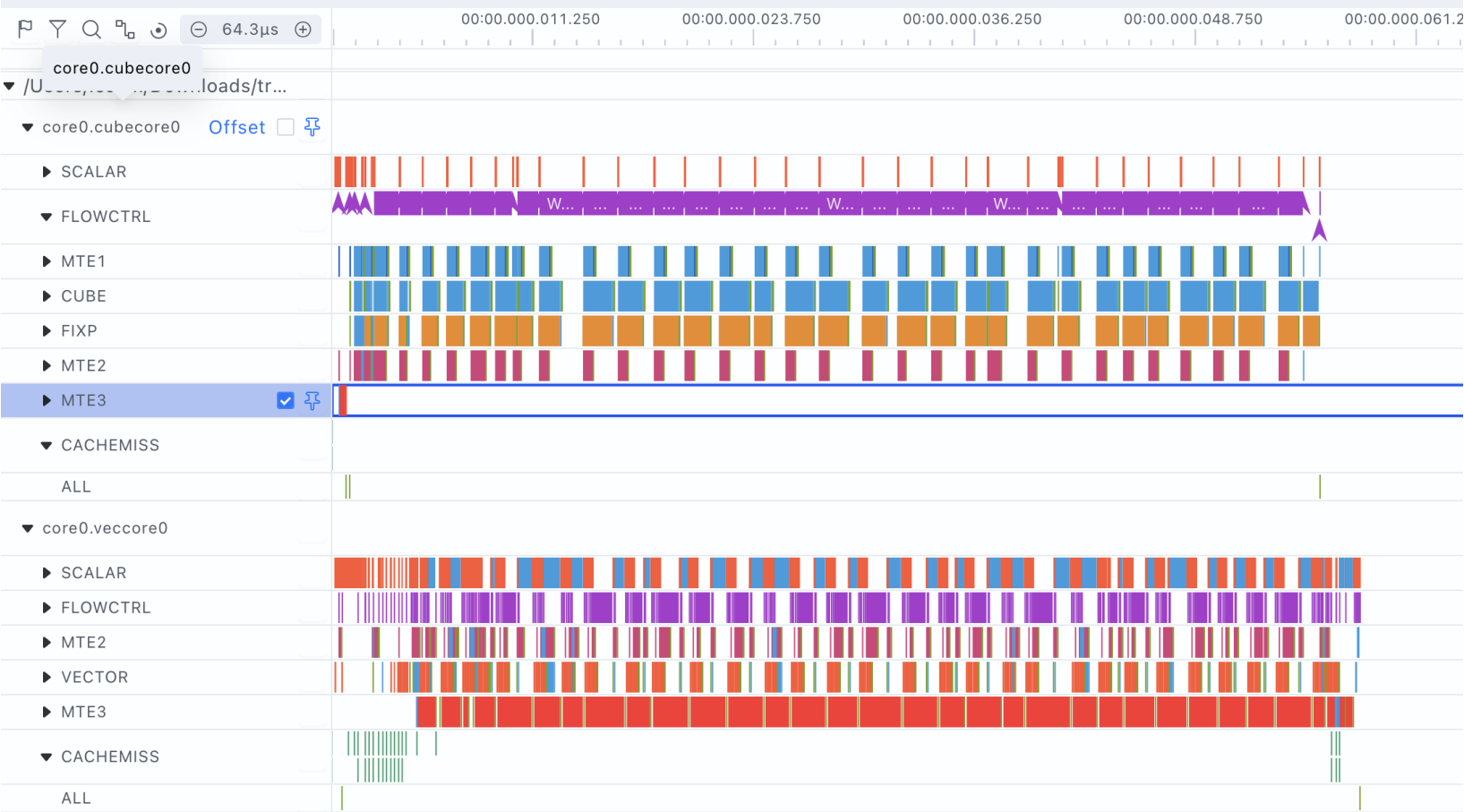
begin = totalTasks * groupId / blockDim;
end   = totalTasks * (groupId + 1) / blockDim;

for (r = 0; r < end - begin; ++r) {
    ordinal = begin + r;           // 当前 group 的连续任务

    kvHead = ordinal / numChunks;
    chunk  = ordinal % numChunks;

    taskId = chunk * Hg + kvHead; // 转回原 task 编号
    ProcessTask(taskId);
}
```

四、瓶颈分析：是否达到极限



UB_to_GM_bw_usage_rate(%)	
NA	
	10.950671
	10.950671

MTE3的带宽仍然也只是极限的11%，至少证明远没有打满MTE3的物理极限，但是MTE3不只受限于其本身，其数据依赖于MTE2的搬入和Vector的计算。

优化方向：进一步优化输出链路！

从流水排列来看，基本已经到达极限



五、一些个人反馈

1、目前可能由于相关CANN赛事多，CANNLAB中的资源有些供不应求，大多时候没有真实的NPU卡能跑本地测试和profile，开发和优化相对困难。

2、是否有可能可以有 ascend c代码的对应的编译后的指令的文件，不然有时在优化的时候，有些灰盒（譬如我之前曾经做过删除pipebarrier的一些实验，有些可有可无的pipebarrier删除后有提速，有些则无；有些应该有数据依赖而不能删除的，删除后竟然无事发生，编译器对ascend c究竟做了什么，相对黑盒）

Thank you.

社区愿景：打造开放易用、技术领先的AI算力新生态

社区使命：以开源驱动技术创新，降低开发门槛，聚力构筑高性能昇腾 AI 算力底座

Vision: Building an Open, Easy-to-Use, and Technology-leading AI Computing Ecosystem

Mission: Drive technological innovation through open source, lower development barriers, and jointly build a high-performance Ascend AI computing infrastructure.



上CANN社区获取干货



关注CANN公众号获取资讯

<https://gitcode.com/cann>

CANN